

ENPH 353 LogBook - Logan Joslin

Sept 16, 2026:

-NOTE: I was not able to access roboTA, so I used chatGPT for help setting up basic video processing functions, as shown below. I designed and implemented the road tracking logic independent of AI.

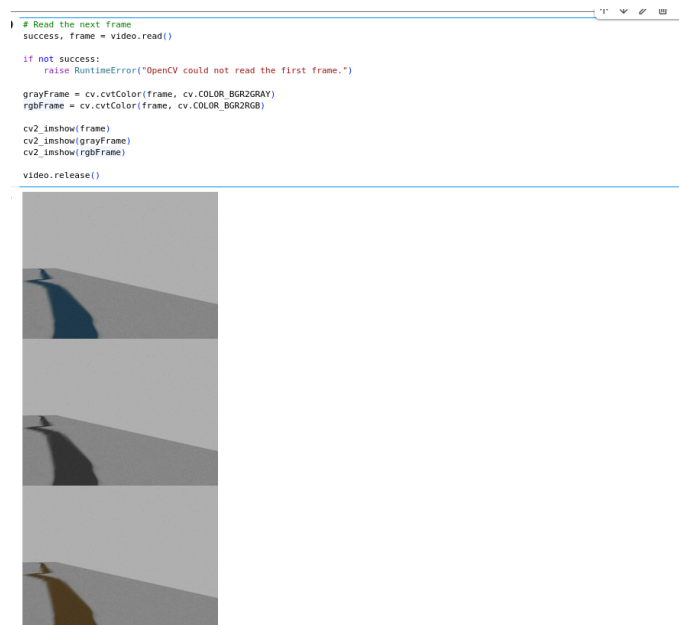
-Setup roboTA according to linux instructions; ran into trouble with student number requirement. Miti said he would alter something on his end to help, but I will resort to and document ChatGPT usage for now.

-Read through OPENCV tutorials since I am already familiar with python, and matplotlib+numpy functions

-With an understanding of all the relevant tools, I used AI to initialize my script, ie: import the video file as an object, and display the functions needed to read a frame

-I noticed that when retrieving fields of the "video" object, many outputs were wrapped in "int". Apparently, the integer datatype is required as an input to many openCV functions.

Next, I experimented with different ways to transform the pixel values in the frame using greyscale and RGB colour inversion:



The grayscale image is obviously easier to process, and still contains a discernible road, so I will work with this.

-Next, I converted the image to a bitmap and separated the road using the cv.threshold() function. Above a certain pixel value threshold, the output image stores a 1, otherwise displays 0.

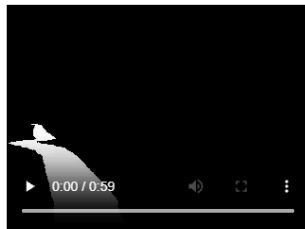
-To help me remember the functionality, I put, in my own words, the meaning of each field and output value of the current functions.

-Finally, I iterated over each frame and applied the bitmap transformation, and exported the results to an output video.

```
from IPython.display import Video, display

# Convert the finished output to a browser-friendly encoding
!ffmpeg -y -loglevel error -i /content/output_video_feed.mp4 -c:v libx264 -pix_fmt yuv420p /content/output_video_browser.mp4

# Display the converted video
display(Video("/content/output_video_browser.mp4", embed=True))
```



-The compression tool (mp4v) that the AI chose is not browser friendly, which made it challenging to display the video. However, the better option (H.264) was not available in this openCV library installation, so I had to separately run (H.264) on FFMpeg in order to display the video.

-In the sample video feed, the red dot seems to remain along the same horizontal axis on the bottom of the frame. This will make my job easier, since I can constrain the final location to the “best” spot on the line, rather than the whole photo.

-I took the arithmetic mean of the white pixel positions along this line and overlaid a vertical line to mark location. It seems to work very well.

-Next, I implemented a basic red circle overlay function. It’s not very efficient, but it works. Lab 2 complete.

Prompts (in order):

I've uploaded a video to google colab into the contents folder and need to process the frames for my lab. Can you help me get set up with OpenCV? Import the necessary files, objects, and functions for reading+writing videos. I will implement the logic and develop an approach. Just get me setup with a generic OpenCV reader and writer.

Worked for 27s

Paste this into a Colab code cell, replacing VIDEO_NAME.mp4 with your uploaded filename. Colab's folder path is /content/.

```
Python

import cv2 as cv
import numpy as np # For your own array processing

# Optional tools for previewing images in Colab
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow

# Input and output file locations
input_path = "/content/VIDEO_NAME.mp4"
output_path = "/content/output_video.mp4"

# Open the input video
video = cv.VideoCapture(input_path)

if not video.isOpened():
    raise FileNotFoundError("Could not open the input video.")

# Read the input video's properties
width = int(video.get(cv.CAP_PROP_FRAME_WIDTH))
height = int(video.get(cv.CAP_PROP_FRAME_HEIGHT))
fps = video.get(cv.CAP_PROP_FPS)
frame_count = int(video.get(cv.CAP_PROP_FRAME_COUNT))

# Choose the output compression format
fourcc = cv.VideoWriter_fourcc("mp4v")

# Create the output video
outputVideo = cv.VideoWriter(
    output_path, # Output file location
    fourcc, # Compression format
    fps, # Same frame rate as the input
    (width, height), # Same dimensions as the input
    isColor=True # Write colour frames
)

if not outputVideo.isOpened():
    raise RuntimeError("Could not create the output video.")

# Read, process, and write one frame at a time
while True:
    success, frame = video.read()

    if not success:
        break # End of video, or a frame could not be read

    # YOUR PROCESSING LOGIC GOES HERE
    # Modify frame before writing it below.

    outputVideo.write(frame)

# Close the input and finish saving the output
video.release()
outputVideo.release()
```

As written, this copies the video frame by frame. Add your processing inside the loop, above `outputVideo.write(frame)`.

- `video.read()` returns a success flag and the next image, `frame`.
- `frame` is a NumPy array with shape `(height, width, 3)`. OpenCV stores its colour channels in blue, green, red order.
- `outputVideo.write(frame)` saves the current frame. Keep its dimensions and colour format consistent with the writer.
- `.release()` closes each video object and finalizes the output file.

Only `cv2` is needed for the reader and writer; the other imports are available for your processing and image

I'm having trouble displaying the new video I've created. I think mp4v compression doesn't work well in my browser. What can I do? Explain your suggestion and alternatives.

Worked for 1m 23s >

Convert your finished video to H.264 using FFmpeg, then display that converted file in Colab. Your suspicion is plausible: `mp4v` has limited browser support, while H.264 is widely supported. An `.mp4` extension describes the file's container—not necessarily the compression format inside it. [MDN's codec guide](#) »

After your video-writing code finishes—including `outputwriter.release()`—run this in a new Colab cell:

```
Python Run

from IPython.display import Video, display

# Convert the output video to a browser-compatible format
!ffmpeg -y -loglevel error -i /content/output_video_feed.mp4 -c:v libx264 -pix_fmt yuv420p /content/output_video_browser.mp4

# Display the converted video
display(Video("/content/output_video_browser.mp4", embed=True))
```

Here's what the conversion command does:

- `-i /content/output_video_feed.mp4` selects the video you already created.
- `-c:v libx264` converts its video compression to H.264.
- `-pix_fmt yuv420p` uses a widely compatible way of storing pixel colours.
- `-y` allows overwriting the converted copy when you rerun the cell.
- `-loglevel error` hides routine messages while still showing errors. [FFmpeg documentation](#) »

The final line embeds the converted video directly into the notebook output, with playback controls.

`embed=True` includes the video data in that output. [IPython documentation](#) »

This uses your existing output video, so you don't need to rerun the image processing. Conversion does take some time and can slightly reduce image quality.

Other options:

Alternative	When it helps
Download and play it in VLC	Useful if you just want to inspect the result on your computer; VLC supports many video formats. VLC »
Create H.264 directly with OpenCV	Avoids the extra conversion, provided your OpenCV installation has an appropriate encoder available. OpenCV documentation »
Convert to WebM using VP8 or VP9	Another option for browser playback, especially if H.264 still causes trouble. Browser codec support »

📄 🔍 ↶ ↷ ⋮